Take-Home Test: Full-Stack LLM Database Chatbot

Build a full-stack application that allows a user to query an attached SQLite database using natural language through a chatbot interface.

The application should include:

1. Frontend

Create a simple chatbot UI where a user can:

- Enter natural language questions about the database
- View the assistant's response clearly
- See returned query results in a readable format, (i.e. single value, table, charts)
- See appropriate error messages when a query cannot be answered

The frontend can be built using any modern framework, such as React, Next.js, Vue, or similar.

2. Backend

Build a backend service that:

- Receives the user's natural language question
- Uses an LLM agent or LLM workflow to translate the question into a SQLite query
- Executes the query safely against the attached SQLite database
- Returns the result to the frontend in a structured format

The backend can be built using Python, for example with FastAPI, Flask, or another suitable framework.

3. SQLite Query Tool Function

Implement a Python tool function for the LLM agent that can query a SQLite database safely and return results in JSON.

Requirements:

- Input: `sqlite_query: str`
- Only allow read-only `SELECT` queries
- Block unsafe or non-read-only operations, including:
  - INSERT
  - UPDATE
  - DELETE
  - DROP
  - ALTER
  - CREATE
  - TRUNCATE

    - REPLACE
    - PRAGMA
    - Multiple statements
- Return JSON containing:
    - ok
    - columns
    - rows
    - row_count
    - truncated
    - error, where applicable
- Enforce a row limit, for example 200 rows
- Handle errors gracefully by returning structured error JSON
- Do not expose raw stack traces to the user

4. Chatbot Behaviour

The chatbot should:

- Answer user questions based only on the attached database
- Ask for clarification when the user's question is ambiguous
- Avoid fabricating answers when the data is unavailable
- Explain briefly what result it found
- Present tabular data clearly where appropriate

5. Safety and Security

The solution should demonstrate safe database access, including:

- Read-only database access
- SQL validation before execution
- Row limits to prevent excessive output
- Protection against SQL injection or prompt-injection style attempts
- No execution of arbitrary code from user input

6. Deliverables

Please submit:

- Source code for the frontend and backend
- Instructions to run the application locally
- Any environment variable setup required
- A short README explaining:
    - Your architecture
    - How the LLM agent/tool flow works
    - Key safety measures implemented
    - Assumptions made
    - Known limitations

7. Evaluation Criteria

Submissions will be assessed based on:

- Correctness and functionality
- Safety of SQL execution
- Quality of full-stack implementation
- Clarity of chatbot user experience
- Code quality and maintainability
- Error handling
- README clarity
- Thoughtfulness of architecture and trade-offs

The attached SQLite database should be used as the data source for the application.